

Politécnico do Porto
Escola Superior de Media Artes e design

ServiGo

Serviços e Interfaces para a Cloud

João Sibrão, João Preto, Filipe Moço

40210416, 40210121, 40210116

Licenciatura em Tecnologias e Sistemas de Informação Para a Web

Introdução

O projeto “ServiGo” nasceu quando o grupo percebeu que faltava uma plataforma prática que ajudasse pessoas a encontrar profissionais para tarefas do dia a dia, desde serviços digitais até pequenos trabalhos presenciais. Apesar da existência de plataformas consolidadas no domínio dos serviços digitais, verificou-se uma ausência de soluções integradas que permitissem a contratação de profissionais para tarefas práticas do quotidiano, como canalização, eletricidade, jardinagem, pintura, pequenas obras ou outros diversos serviços. O objetivo principal foi, portanto, desenvolver um ecossistema que centralizasse estes dois tipos de serviços num único espaço, proporcionando ao utilizador uma experiência simples, fiável e transparente.

O modelo de negócio assenta na subscrição mensal dos profissionais que desejam manter o seu perfil ativo na plataforma. Em vez de taxas por contratação, optou-se por um sistema previsível e estável, onde o freelancer ou a empresa pode apresentar o seu portefólio, serviços oferecidos, certificações, preços e avaliações recebidas. Para o cliente, a plataforma funciona como um ponto central onde pode pesquisar, comparar, avaliar e contactar profissionais de forma rápida, clara e organizada. A transparência e a autonomia na negociação foram prioridades desde o início, evitando a interferência direta da plataforma nas transações.

Sumário

Introdução.....	2
Descrição do Sistema	4
Requisitos Funcionais e Não Funcionais da Plataforma	4
1. Requisitos Funcionais	4
1.1 Gestão de Identidade.....	4
1.2 Catálogo de Serviços	4
1.3 Gestão de Perfil.....	5
1.4 Reservas (Booking).....	5
1.5 Avaliações	5
Requisitos Não Funcionais e Infraestrutura	5
Descrição dos Microserviços da Plataforma “ServiGo”	6
1. API Gateway.....	6
2. Auth Service	7
3. User Service	7
4. Catalog Service.....	8
5. Booking Service.....	8
Desafios e Benefícios.....	9
6. Trabalho Futuro.....	9
Anexos	10
Diagrama de arquitetura.....	10
Link do GitHub	10

Descrição do Sistema

A plataforma foi concebida com base numa arquitetura de microserviços, devido à sua capacidade de permitir uma separação clara de responsabilidades e à facilidade de escalar cada área consoante as suas necessidades. Cada microserviço é responsável por um domínio específico: autenticação, gestão de utilizadores, catálogo de serviços e sistema de reservas (futuramente ainda serem propostos a criação de mais microserviços). Esta divisão facilita o desenvolvimento paralelo, reduz o acoplamento e melhora a robustez geral da aplicação.

A comunicação entre microserviços é realizada de forma síncrona através de APIs REST, sendo todo o tráfego externo encaminhado por um API Gateway que valida pedidos e centraliza o acesso ao sistema.

Requisitos Funcionais e Não Funcionais da Plataforma

Nesta secção apresento os principais requisitos funcionais e não funcionais definidos para o desenvolvimento da plataforma *ServiçosJá*. Estes requisitos orientam toda a arquitetura do sistema e garantem que o produto final responde às necessidades reais dos utilizadores.

1. Requisitos Funcionais

1.1 Gestão de Identidade

A plataforma precisa de permitir o registo e autenticação tanto de clientes como de profissionais. Isto inclui criar conta, fazer login e garantir que cada utilizador tem acesso apenas às áreas que lhe dizem respeito. É através desta base que os restantes serviços conseguem identificar quem está a usar o sistema.

1.2 Catálogo de Serviços

Os utilizadores devem conseguir pesquisar e consultar serviços disponíveis, aplicando filtros como preço, localização ou categoria. A ideia é facilitar ao máximo a descoberta dos profissionais adequados, tornando o processo rápido e intuitivo.

1.3 Gestão de Perfil

Cada utilizador tem acesso a um perfil pessoal que pode atualizar quando quiser. No caso dos profissionais, este perfil inclui também a possibilidade de adicionar fotografias de trabalhos anteriores, escrever descrições e indicar horários de disponibilidade. Isto ajuda os clientes a avaliarem melhor a qualidade e profissionalismo de quem pretendem contratar.

1.4 Reservas (Booking)

O processo de reserva é um dos pontos centrais da plataforma. Os clientes podem solicitar um serviço, e os profissionais têm a possibilidade de aceitar ou rejeitar o pedido. Além disso, tanto clientes como profissionais conseguem consultar o histórico de serviços prestados ou recebidos.

1.5 Avaliações

Após a conclusão de um serviço, o cliente pode deixar uma avaliação e um comentário sobre o profissional. Este sistema de rating ajuda a criar confiança na plataforma e permite destacar os profissionais com melhor desempenho.

Requisitos Não Funcionais e Infraestrutura

O sistema foi projetado para garantir alta disponibilidade e escalabilidade, recorrendo a serviços stateless que podem ser replicados de forma independente. A infraestrutura assenta em Docker, permitindo uniformidade entre ambientes de desenvolvimento e execução. O uso de bases de dados independentes para cada microserviço reduz conflitos, melhora o desempenho e permite o uso de diferentes tecnologias conforme o tipo de dados.

Descrição dos Microserviços da Plataforma “ServiGo”

1. API Gateway

O API Gateway é o ponto de entrada principal de toda a plataforma *ServiçosJá*. Sempre que a aplicação Web ou Mobile faz um pedido, é o gateway que o recebe primeiro e decide para que microserviço esse pedido deve ser encaminhado. Isto permite que os clientes não tenham de lidar diretamente com a estrutura interna da aplicação, tornando o sistema mais seguro e mais simples de utilizar.

Além do encaminhamento, este serviço trata também de questões essenciais de segurança. Aqui são aplicados middlewares globais como CORS ou Helmet, e é também feito o controlo básico de autenticação, verificando se o utilizador enviou um token válido antes de permitir o acesso a rotas protegidas. É uma espécie de “porteiro” da aplicação: se o utilizador não estiver autenticado ou faltar alguma validação, o pedido nem chega aos restantes serviços.

Embora neste momento o gateway esteja focado principalmente em roteamento e segurança, ele pode evoluir para um papel mais avançado no futuro, incluindo agregação de respostas de vários microserviços. Isso evitaria que o frontend fizesse múltiplas chamadas separadas, tornando a aplicação mais eficiente e rápida.

2. Auth Service

O Auth Service é o responsável por tudo o que está relacionado com autenticação e gestão de identidades dos utilizadores. Sempre que alguém se regista na plataforma, é este serviço que trata da criação da conta, incluindo o hashing das passwords para garantir que são armazenadas de forma segura. Quando o utilizador faz login, é o Auth Service que valida as credenciais e gera um token JWT, que depois será usado para provar que o utilizador está autenticado enquanto percorre a aplicação.

Este serviço também disponibiliza um endpoint interno que permite ao API Gateway confirmar se um token enviado é realmente válido. Assim, garante-se que cada acesso a funcionalidades protegidas passa sempre pela lógica centralizada deste microserviço. Em termos de dados, o Auth Service armazena apenas as informações essenciais: emails, passwords encriptadas e o tipo de utilizador (por exemplo, cliente ou prestador de serviços).

3. User Service

O User Service complementa o Auth Service ao gerir todas as informações detalhadas sobre os utilizadores que não estão diretamente relacionadas com autenticação. Enquanto o Auth Service guarda apenas as credenciais, é o User Service que armazena tudo aquilo que compõe o perfil do utilizador dentro da plataforma — nome, fotografia, descrição, localização, portfólio e até avaliações deixadas por outros utilizadores.

Como este serviço lida com dados muito variados e flexíveis, foi utilizado MongoDB como base de dados. Para facilitar o trabalho do frontend, o serviço foi implementado com GraphQL, permitindo que cada componente do cliente peça apenas os campos de que realmente precisa. Por exemplo, numa listagem de profissionais o sistema pode pedir apenas nome e fotografia, enquanto na página de perfil o pedido inclui todos os detalhes disponíveis.

4. Catalog Service

O Catalog Service é responsável por organizar e gerir toda a oferta de serviços disponível dentro da plataforma. É aqui que são criadas as categorias — como limpeza, eletricidade, jardinagem ou construção — e é também onde os profissionais registam os serviços que prestam, indicando informações como preço, descrição e especializações.

Este microserviço também gere toda a lógica de pesquisa e filtragem dos serviços. Assim, quando um utilizador procura por algo específico, como “eletricista no Porto” ou “trabalhos de jardinagem abaixo de 25€”, é o Catalog Service que responde, devolvendo a lista adequada. Como os dados aqui são muito estruturados e com relações claras entre si, a escolha natural para a base de dados foi MongoDB

5. Booking Service

O Booking Service é o microserviço responsável por todo o fluxo de reservas dentro da plataforma. Sempre que um cliente escolhe um serviço e pretende avançar com a contratação, é este serviço que regista a reserva e acompanha todo o processo até à conclusão.

Para controlar o estado das reservas, o Booking Service utiliza uma máquina de estados. Assim, cada reserva passa por várias etapas: começa normalmente em *PENDING* enquanto aguarda resposta do profissional; depois pode ser *CONFIRMED* se o profissional aceitar, ou *REJECTED* se recusar; por fim, quando o serviço for realizado, o estado é atualizado para *COMPLETED*. Esta lógica ajuda a manter o sistema organizado e evita ambiguidades sobre o estado de cada pedido.

O Booking Service precisa também de comunicar com outros microserviços. Contacta o User Service para confirmar dados do cliente e do prestador, e consulta o Catalog Service para garantir que o serviço existe e que o preço está correto. Como lida com dados muito flexíveis e eventos que podem mudar rapidamente, utiliza uma base de dados MongoDB.

Desafios e Benefícios

A abordagem baseada em microserviços trouxe claras vantagens, como a facilidade de manutenção, escalabilidade e capacidade de atualizar componentes de forma isolada. No entanto, introduziu também desafios adicionais que o grupo encontrou ao longo do desenvolvimento, particularmente ao nível da comunicação entre serviços, gestão de falhas e consistência dos dados. A aplicação de técnicas de observabilidade, logging e monitorização revelou-se essencial para garantir o funcionamento correto do sistema e a deteção rápida de problemas.

6. Trabalho Futuro

A primeira entrega do projeto concentrou-se essencialmente na definição da arquitetura e na configuração da infraestrutura da plataforma *ServiGo*. Com a base de microserviços já estruturada e os princípios de comunicação entre os serviços estabelecidos, as próximas fases de desenvolvimento irão focar-se na implementação detalhada das funcionalidades e na integração completa do sistema.

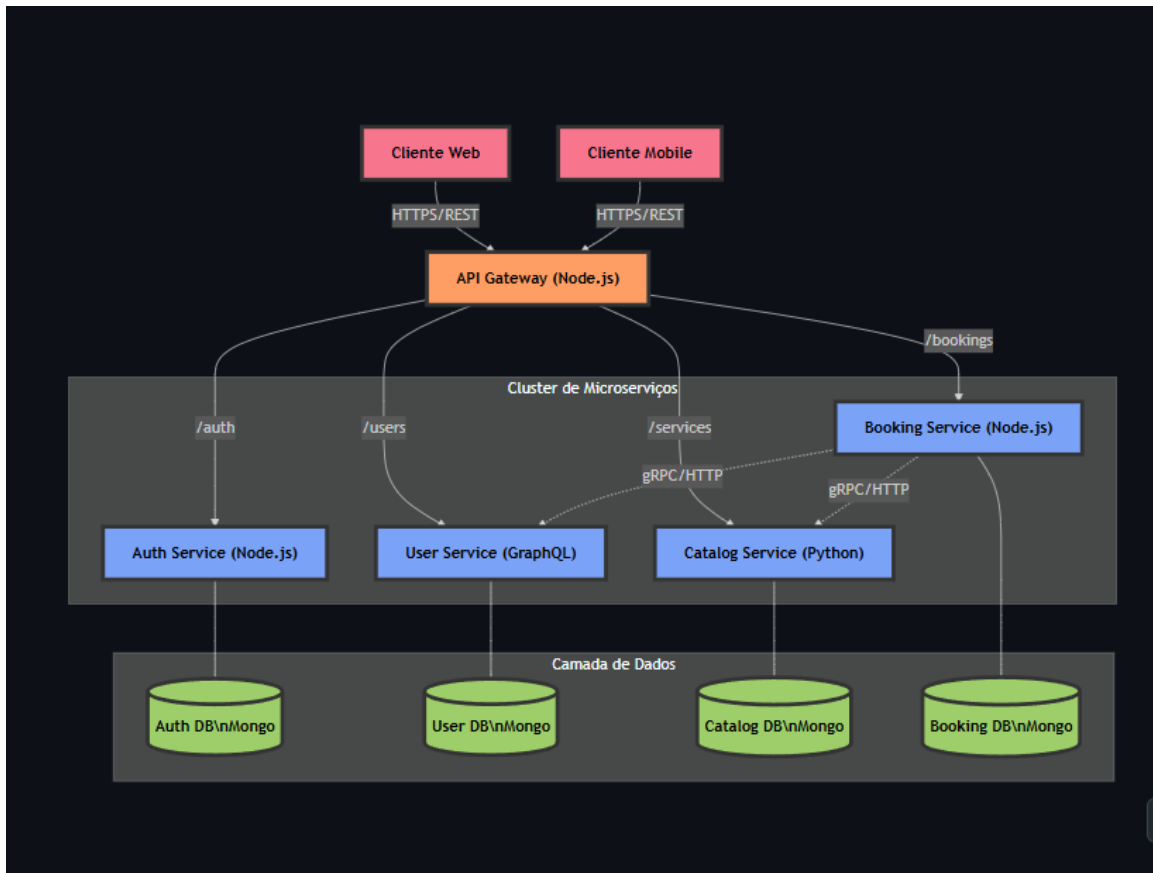
Um dos pontos centrais será a persistência de dados, que envolve a criação de conectores para as bases de dados PostgreSQL e MongoDB, bem como a definição dos esquemas ou modelos de dados necessários para armazenar informação sobre utilizadores, serviços e reservas. Paralelamente, será implementada a lógica de negócio de cada serviço, incluindo as regras de validação, cálculos de preços e a gestão dos ciclos de vida das reservas através de máquinas de estado.

Outro objetivo será assegurar a comunicação eficiente entre os microserviços, tornando efetivas as chamadas entre Booking, User e Catalog services, seja através de HTTP ou gRPC, garantindo que os dados circulam corretamente e que as ações de um serviço se refletem nos restantes de forma consistente. Finalmente, será desenvolvido o frontend da plataforma, proporcionando uma interface intuitiva e funcional tanto para clientes como para profissionais, de forma a tornar a utilização do sistema simples e agradável.

Este trabalho futuro representa o passo lógico seguinte após a construção da arquitetura, permitindo transformar o projeto conceptual numa aplicação funcional e robusta, pronta para testes reais e utilização prática.

Anexos

Diagrama de arquitetura



[Link do GitHub](#)